

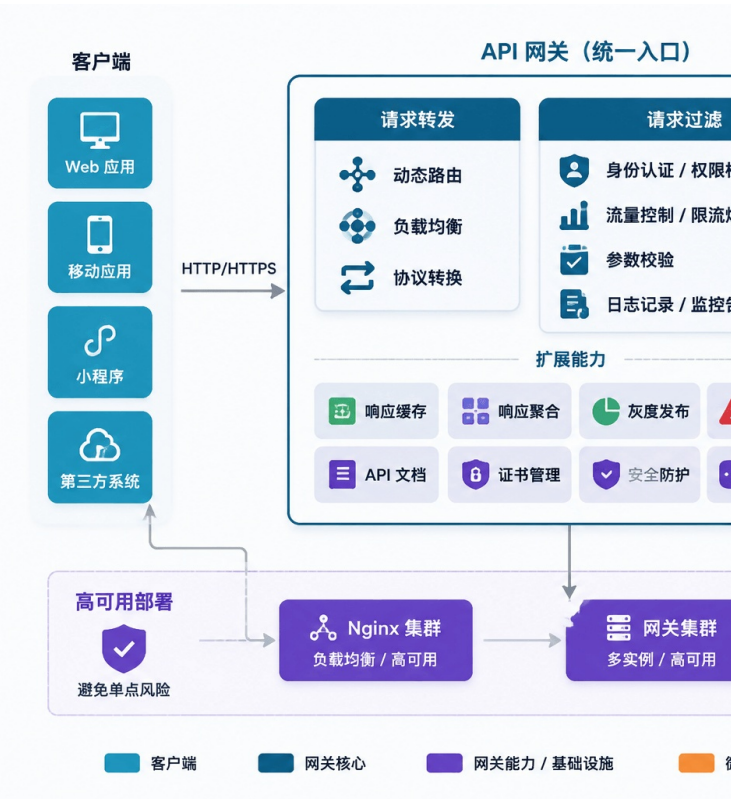
大模型网关详解：多模型路由、Fallback、限流与成本控制

大模型网关基础

LLM Gateway到底是什么？

LLM Gateway 更像是：API 网关能力 + 模型调用控制面。

传统 API 网关是位于客户端与后端服务之间的**统一入口**，所有客户端请求先经过网关，再由网关路由到具体的目标服务，主要管 HTTP 流量：鉴权、限流、转发、日志、熔断。



传统 API 网关示意图

LLM Gateway 则面对的是大模型调用，它除了处理普通 API 问题，还要处理模型特有的问题：模型选择、Token 预算、上下文长度、供应商差异、流式输出、工具调用、结构化响应、成本统计、Prompt 版本和输出质量。

更准确地说，LLM Gateway 是应用层和模型供应商之间的一层治理入口。它不一定替代企业已有的 API 网关，但会把模型调用相关的路由、预算、审计和适配逻辑收口。



业务代码不直接关心 OpenAI、Anthropic、Gemini、Qwen、DeepSeek、私有化模型分别怎么调，而是统一向 Gateway 发一个标准请求。Gateway 根据场景、预算、延迟、模型可用性和业务策略，决定调用哪个模型、走哪个供应商、是否需要重试、是否需要降级、怎么记录日志。

第一版 Gateway 可以很轻，只做统一封装、超时、重试和日志。到生产阶段，它通常还会管理模型路由、Token 预算、限流、成本归因、缓存、审计和安全策略。

如果只做“把请求转发一下”，它只是一个代理；开始记录为什么选这个模型、怎么扣预算、失败后怎么兜底，才进入 Gateway 的范围。

为什么需要 LLM Gateway？

很多团队第一次做 AI 应用时，会直接在业务服务里写模型调用：

```
Controller -> Service -> OpenAI SDK -> 返回答案
```

这条链路很短，开发体验也好。但只要线上规模稍微起来，问题会集中暴露。

直连模型的典型问题	线上表现	Gateway对应能力
模型名写死	模型升级、下线、切换供应商时到处改代码	模型注册表 + 配置化路由
API Key 分散	多个服务各自保存密钥，轮换困难	统一密钥管理
供应商限流	429 后业务服务疯狂重试，越重试越糟	限流、排队、Fallback、熔断
成本不可见	月底只知道总账单，不知道哪个租户、功能、Prompt 花钱	usage 记录 + 成本归因
所有请求走同一模型	简单任务浪费钱，复杂任务效果差	按任务类型做模型路由
日志缺失	用户投诉“刚才 AI 胡说”，排查时找不到模型输入输出	Trace、Prompt 版本、模型调用日志
供应商 SDK 分散	每个业务都处理流式、错误码、重试和结构化解析	Provider Adapter 统一封装

这里最容易被低估的是成本和排查。

传统 API 调用失败，通常能从状态码、请求参数、数据库状态里定位。LLM 调用失败就麻烦得多：可能是 Prompt 版本变了，可能是模型升级了，可能是检索上下文噪声太多，可能是输出被截断，可能是路由去了一个便宜但能力不够的模型。

没有 Gateway，所有这些线索都散在业务系统里。散了就很难管。

LLM Gateway 和 LLM Router 有什么区别？

Router 管的事情比较窄：这个请求该选哪个模型。输入是用户问题、任务类型、预算、上下文长度这些，输出就是一个模型名或者一组候选。

Gateway 的范围大得多。从请求进入到结果返回，中间经过的鉴权、限流、路由、fallback、日志、成本记录，都归它管。Router 只是 Gateway 里的一个环节。

维度	LLM Router	LLM Gateway
主要职责	模型选择	统一接入、路由、限流、Fallback、观测、成本治理
决策粒度	单次请求选模型	请求全生命周期治理
典型输入	用户问题、任务类型、预算、上下文长度	请求、用户、租户、场景、Prompt、模型、供应商、策略
典型输出	目标模型或模型集合	完整调用结果、usage、日志、错误、成本、Fallback 轨迹
适合阶段	多模型调用开始变复杂	AI 应用进入生产

可以这么理解：**Router 负责选模型，Gateway 负责把整个模型调用管起来。**

你可以只有 Router，没有 Gateway，就做简单的模型路由功能。例如写一个函数，根据任务类型返回对应的模型。这能解决一部分成本问题，但解决不了密钥管理、限流、日志、审计、统一错误处理和供应商切换。

反过来，一个早期 Gateway 也可以先没有复杂 Router。第一版只做统一接入、日志和 Fallback，就已经能减少很多生产事故。

真正落地时，路由策略不要绑死在某个具体模型名上，而要尽量绑定到模型层级、成本区间、上下文能力、风险等级这些更稳定的属性。模型会升级，名字会变，但这些决策维度不会消失。

LLM Gateway 和 RAG、Agent、MCP 是什么关系？

这几个概念经常一起出现，但边界不一样。

概念	主要解决什么问题	和 Gateway 的关系
RAG	检索外部知识，把相关上下文塞进模型请求	Gateway 可以限制 Token、记录 Prompt 版本、缓存检索后结果，但不负责检索质量本身
Agent	拆解任务、调用工具、多轮执行	Gateway 可以管理每一步模型调用的预算、路由和 Fallback，不决定 Agent 的任务规划逻辑
MCP	注册模型或 Agent 以统一协议访问工具、资源和上下文	Gateway 可以审计和治理模型请求，也可以配合工具调用日志，但不替代 MCP Server 或工具注册表

所以，Gateway 更靠近“模型调用治理”；RAG、Agent、MCP 更靠近“应用能力组织”。一个复杂 Agent 可以在多个步骤里调用 Gateway，Gateway 也可以对每个步骤分别记录 scene、route_reason、Token 使用量和成本。

LLM Gateway 会不会增加延迟？

会。

任何中间层都会增加一点处理时间。问题是这点时间到底值不值。

如果 Gateway 只是在同机房里做一次内存路由、Token 估算和日志写入，额外延迟就还好。真正吃时间的是模型侧：模型排队、长上下文推理、跨区域网络、输出 Token 过多、工具调用链路拉长、以及重试。

Gateway 反过来还能帮你把端到端延迟压下来：

- 简单任务直接丢给小模型，不用每次都排大模型的队。
- 重复问题走缓存，模型都不用调。
- 语音交互、在线客服这类延迟敏感的场景，优先选 TTFT 更稳定的模型或供应商。
- 供应商抖动时快速 Fallback，别让用户干等到超时。
- 上下文太长的请求提前压缩，省得模型端慢慢算。

这里有个边界：第一版 Gateway 不要写成“每次请求都调用一个强模型做路由判断”。路由本身也会消耗 Token 和时间，如果没有足够流量、评测集和质量反馈，很容易把省下来的钱花在路由判断上。

第一版从规则和轻量分类开始，通常更划算。

你真的需要 LLM Gateway 吗？

很多项目一开始用不上完整的 LLM Gateway。

Gateway 解决的是规模化之后的问题：多团队共用模型、多供应商切换、成本精细归因、合规审计留痕。如果你的场景还没长到这个阶段，提前搭完整 Gateway 只会增加维护负担，不会带来收益。

如果你只是做内部工具、单模型、低流量、没有多租户、没有严格成本压力，也不需要复杂审计，那就先别过度设计。一个封装良好的 LLMClient，加上基础日志、超时、重试和错误处理，已经够用了。

判断方式也不复杂，按你的实际情况对号入座：

- 只有一个应用、一个模型、每天几百次调用 → 先不用 Gateway，把精力花在业务逻辑上。
- 有多个业务线或团队都在调用模型 → 开始收口，统一入口和调用规范。
- 有多租户、配额管理、成本需要按团队或场景归因 → 需要 Gateway。
- 有多供应商、需要 Fallback、想做模型路由 → 需要 Gateway。
- 有合规审计、Prompt 版本管理、线上质量回放、敏感内容拦截 → 必须 Gateway 化。

工程里不怕第一版简单，怕的是简单到没有边界。

你可以先不做完整 Gateway，但最好从第一天就把模型调用收在一个地方——一个统一的模块、一个统一的接口。后面不管是加日志、加限流、加路由、还是换供应商，改的都是这一个点，而不是满仓库 grep。

为什么不能所有请求都用最强模型？

最贵的模型不一定是最适合的模型

有些团队一开始会默认选最贵最强的模型，觉得多花点钱可以换稳定性。

对高价值、强推理、高风险任务，强模型确实值得。但所有请求都走强模型，很快会遇到三个问题：

1. **成本不可控**：分类、改写、摘要这类任务也走旗舰模型，单次看不贵，流量上来后很吓人。
2. **延迟不稳定**：强推理模型为了复杂任务设计，不一定适合实时对话、语音交互、轻量判断。
3. **资源被浪费**：简单任务没有给强模型发挥空间，复杂任务反而可能因为上下文组织差而答不好。

以一组常见的内部模型分层为例：**tier-fast** 更适合低成本、快响应场景，**tier-pro** 更适合复杂推理和高质量输出。具体价差不要写死在业务代码里，应该放在模型注册表或价格快照里维护。

Gateway 在这里解决的不只是钱，还有后续换模型、控延迟、查问题时的混乱：**它要根据质量、成本、延迟做取舍，而不是固定选一个最强模型。**

什么任务适合小模型？什么任务必须上强模型？

实际落地时，可以先把任务分成三层：

第一层：能不用大模型就不用。

比如固定规则过滤、关键词判断、权限校验、简单模板填充，这些交给代码更稳定。别让模型去判断“用户是不是空字符串”“文件后缀是不是 PDF”。

第二层：能用小模型就先用小模型。

典型场景是意图分类、轻量摘要、标题生成、简单改写、低风险信息抽取。这类任务更需要结构化输出、枚举约束和失败兜底，不一定需要旗舰模型。

第三层：复杂任务再升级。

多文档归纳、代码架构设计、复杂 Agent 规划、强事实核验、金融法务医疗相关内容，错误成本高，强模型更合理。

LLM Router 如何选择模型？

LLM Router 的任务，是给每个请求选一个合适模型。

这里的合适不只看回答质量，还要看成本、延迟、上下文长度、供应商可用性和风险策略。

LLMRouter 这类智能路由项目，思路是为每个查询动态选择更合适的模型，从而在质量、成本和延迟之间做取舍。它覆盖了单轮路由、多轮路由、个性化路由、Agentic 路由等方向，也提供 KNN、SVM、MLP、Matrix Factorization、Elo Rating、Graph-based routing 等策略。

这些策略适合学习和实验，但生产里要先解决可解释性和回放能力。更稳的路线是：**模型路由从简单规则出发，然后根据实际场景慢慢演进成可训练、可评估、可迭代的系统。**

常见路由策略有这几类：

** 路由策略 **	** 怎么做 **	** 适合场景 **	** 风险 **
固定规则路由	按业务场景、接口、租户套餐选择模型	第一版 Gateway，大多数业务足够用	规则维护靠人，容易滞后
成本优先 / 级联路由	默认走便宜模型，失败或低置信度再升级	分类、摘要、客服 FAQ	低成本模型误判会传导
语义 / 分类路由	根据 Query 语义、复杂度、风险等级选择模型	问题类型稳定、流量较大	阈值和分类器需要持续调优
学习型路由	基于历史质量、成本、延迟训练 Router	多模型、多任务、大流量	依赖评测数据和反馈闭环
个性化路由	结合用户偏好、历史交互选择模型	C 端助手、教育、内容平台	隐私和一致性成本更高
Agentic 路由	多轮任务里动态切换模型和工具	复杂 Agent、长链路由任务	调试和成本控制难度高

第一版别急着上复杂 Router。

固定规则路由是起步首选。直接按任务、用户类型、请求信息指定对应模型，好落地、结果可控；代价是规则要人工维护，业务调整后容易滞后。做第一版网关时，这个取舍通常可以接受。

举个很容易理解的例子：翻译任务走模型 A、代码生成走模型 B、默认走模型 C。免费用户走小模型，付费用户走强模型。

成本优先 / 级联路由 (Cascade) 是规则路由之后常见的进阶方式：先让小模型处理，解析失败、置信度低、质量校验不通过时再升级到更强模型。

难点在于什么时候判断小模型不够用。它还会增加一次模型推理或评估，只有在应用能容忍额外延迟的场景下才可行。

语义 / 分类路由把路由决策从硬编码规则升级到基于内容理解。常见做法是把 Query 编码成 embedding，再和任务原型、模型 profile 或参考 prompt 做相似度匹配；也可以用轻量分类器判断请求复杂度和风险等级。

风险在于 embedding 模型本身的选择和更新是维护成本，分类器会随着业务变化和 query 分布漂移而退化，需要定期用新数据重新评估阈值和重训模型。

学习型路由门槛最高，得靠充足的标注、质量、成本和延迟数据训练。没有评测集和线上反馈，模型选择器很难解释，出了问题也不好复盘。

在此基础上的**个性化路由**，按用户习惯适配模型，适合 C 端，但隐私和调试难度大。

Agentic 路由是更复杂的一类方向。在多轮 Agent 任务中，路由不再是一次性决策——Agent 在执行过程中可能需要在不同步骤调用不同模型（比如规划步骤用强模型、工具调用用快模型、总结步骤用便宜模型），甚至需要根据中间结果动态调整后续步骤的模型选择。

LLM Gateway 需要具备哪些能力？



LLM 网关示意图

多模型统一接入

业务代码里最不该到处散落的，就是供应商 SDK 调用。今天一个服务调 OpenAI，明天另一个服务调 DeepSeek，后天一个定时任务又接了 Gemini。短期看都能跑，时间一长就会变成一堆重复逻辑：API Key、超时、重试、流式解

析、错误码、usage、日志格式、模型名映射，每个地方都处理一遍。

更稳的做法，是先定义统一请求和响应。

```
1 from dataclasses import dataclass, field
2 from typing import Any, AsyncIterator, Protocol
3
4 @dataclass
5 class LLMRequest:
6     request_id: str
7     idempotency_key: str
8     tenant_id: str
9     user_id: str
10    scene: str
11    messages: list[ChatMessage]
12    response_schema: dict[str, Any] | None = None
13    options: LLMOptions | None = None
14
15 @dataclass
16 class LLMResponse:
17     request_id: str
18     model: str
19     provider: str
20     content: str
21     usage: TokenUsage
22     finish_reason: str
23     fallback_used: bool = False
24
25 class ProviderClient(Protocol):
26     """供应商适配器：一个供应商一个实现，负责协议翻译。"""
27
28     def provider_name(self) -> str: ...
29
30     def supports(self, model: str) -> bool: ...
31
32     def chat(self, request: LLMRequest, prompt: RenderedPrompt,
33     ↪ route: ModelRoute) -> LLMResponse: ...
34
35     async def stream_chat(
36         self, request: LLMRequest, prompt: RenderedPrompt, route:
37         ↪ ModelRoute
38     ) -> AsyncIterator[LLMChunk]: ...
39
40 class LLMGateway(Protocol):
41     """业务侧只依赖这一个门面，不直接接触供应商 SDK。"""
42
43     def chat(self, request: LLMRequest) -> LLMResponse: ...
```

这几个接口解决几个实际问题：

- 业务侧只依赖 LLMGateway，不依赖某个供应商 SDK。
- 模型名、供应商、fallback 策略都能配置化。
- usage、成本、错误、延迟可以统一记录。
- 后续接入新模型，只需要增加 Provider Adapter。

统一请求的入口形状，工程上常见的是 OpenAI Chat Completions 兼容风格。LiteLLM、DeepSeek、Qwen 等方案都提供了类似入口，Kong AI Gateway 这类网关也会用 OpenAI 兼容格式作为 AI 插件的通用入口之一。

对外暴露 OpenAI 兼容接口的好处很直接：业务方通常不用大改 SDK，改 base_url 或网关地址就能从直连供应商切到统一入口。

但这只是入口形状统一，不代表出口也统一。Cloudflare AI Gateway 这类托管网关还要按它当前文档支持的 Provider Native、REST 或 Binding 集成方式接入，不能默认所有供应商都能被当成同一个 OpenAI 协议透传。OpenAI 协议也表达不了一些供应商的专属能力，比如 Anthropic 的 extended thinking、Gemini 的 grounding 元数据。这类能力通常要放进 extra_body、metadata 或内部扩展字段里，再由 Provider Adapter 转成目标供应商自己的请求格式。

Provider Adapter 真正难的也在这里：endpoint 和鉴权头只是最外层，工具调用、流式事件、系统提示、结构化输出、usage 和错误码都要翻译对。

维度	OpenAI Chat Completions	Anthropic Messages API	Gemini generate-Content
工具调用字段	tool_calls	tool_use content block	functionCall part
工具结果回传	role=tool 消息	role=user + tool_result content block	functionResponse part
工具 Schema	JSON Schema	JSON Schema 子集	OpenAPI 子集
系统提示位置	messages 中的 system/developer	顶层 system 字段	systemInstruction
多工具调用	原生支持	原生支持	结合模型和 SDK 行为单独验证
专属能力扩展	metadata / 扩展参数	thinking、cache_control 等	grounding、cachedContent 等

所以，OpenAI 兼容更像是业务侧的“门面协议”。门面后面要不要支持 Claude、Gemini、私有模型，工作量主要落在 Provider Adapter 上。产品化网关也一样，官方文档写着支持某类 Provider，不等于所有模型能力都能无损映射。

第一版不用追求“大而全”。先把模型调用收口，后面再补路由、限流和审计。

模型路由

模型路由很容易看到收益，尤其是有明显任务分层的系统。第一版可以配置化，不需要训练模型。

```
1 routes:
2   - scene: intent_classification
3     primary: tier-fast
4     fallback:
5       - tier-nano
6       - tier-balanced
7     max_output_tokens: 256
8     risk_level: low
9
10  - scene: complex_reasoning
11    primary: tier-flagship
12    fallback:
13      - tier-pro
14      - tier-balanced
15    max_output_tokens: 4096
16    risk_level: medium
17
18  - scene: legal_review
19    primary: tier-flagship
20    fallback:
21      - tier-compliance
22    require_human_review: true
23    risk_level: high
24
25 default:
26   primary: tier-balanced
27   fallback:
28     - tier-fast
```

这里的 tier-* 是网关内部的模型层级名，不是供应商真实模型 ID。生产里通常会由 Model Registry 把 tier-fast、tier-balanced、tier-flagship 映射到当前可用的具体模型，并且在日志里同时记录“模型层级”和“真实模型名”。这样模型升级时只改注册表和灰度配置，不用改业务路由规则。

路由决策时，Gateway 至少要看这些因素：

因素	作用
scene	业务场景，决定默认模型和风险等级
输入 Token	判断是否超过模型上下文窗口或预算
输出长度	控制成本和延迟
用户套餐	免费用户和企业用户可以走不同模型
风险等级	高风险任务强制走合规模型或人工审核
当前模型状态	供应商异常、429、P95 延迟升高时切走
历史质量	某模型在某类任务上持续失败时降低权重

一个简单路由器可以先这样写：

```
1 class RuleBasedModelRouter:
2     """规则路由：只负责选模型，不负责调模型。"""
3
4     def __init__(
5         self,
6         route_config_repository: RouteConfigRepository,
7         model_health_service: ModelHealthService,
8     ) -> None:
9         self._route_config_repository = route_config_repository
10        self._model_health_service = model_health_service
11
12    def route(self, request: LLMRequest, budget: TokenBudget) ->
13    ModelRoute:
14        policy = (
15            self._route_config_repository.find_by_scene(request.
16            scene)
17            or self._route_config_repository.default_policy()
18        )
19
20        for model in policy.candidates:
21            if not budget.fits(model):
22                continue
23            if not self._model_health_service.is_available(model):
24                continue
25            return ModelRoute.of(model, policy.provider_of(model),
26            policy)
27
28        raise NoAvailableModelError(request.scene)
```

这段代码不复杂，重点在职责边界：路由器只负责选模型，不负责调模型；健康检查只提供状态，不掺业务逻辑；预算判断单独放出来，后续替换估算方式也方便。

优雅降级

Fallback不是“失败就换一个模型再试”这么简单。首先要区分错误类型。

错误类型	是否适合 Fallback	处理方式
网络瞬断	适合	短重试后切备用模型
供应商 5xx	适合	重试 + 熔断 + 切供应商
429 限流	适合但要谨慎	读 Retry-After，必要时排队或切模型
上下文超限	不适合直接重试	压缩上下文、减少检索片段或换长上下文模型
参数错误	不适合	修请求，不要重复打供应商
安全拒答	通常不适合	进入业务拒答或人工流程
结构化解析失败	可有限修复	让模型修 JSON 或降级 Schema

一个 Fallback 链可以写成这样：

```
1 优先模型可用 -> 正常调用
2 优先模型 429 -> 读取限流信息 -> 切备用同级模型
3 备用模型也不可用 -> 切轻量模型并缩短输出
4 仍不可用 -> 排队、返回降级提示或转人工
```

这里有两个容易被忽略的细节。

第一，Fallback 必须和幂等绑定。用户点一次“生成报告”，主模型已经生成完了，但你的网关超时了，于是又切备用模型生成一次，最后落库两份报告，成本也扣两遍。

第二，Fallback 不能偷偷改变业务语义。法务审核任务从强模型降到便宜模型，如果不标记、不审核，很容易把风险藏起来。高风险场景里，宁愿返回“当前系统繁忙，稍后重试”，也不要硬给一个低质量答案。

幂等键是 LLM 高消费场景的兜底护栏。它和普通 HTTP 幂等不完全一样：HTTP 幂等通常关注“重复执行后资源状态一致”，但 LLM 重复执行会多扣一次钱，而且输出可能变成另一个版本。所以 Gateway 侧不能只存一个“已处理”标记，最好把最终 LLMResponse 也按 tenant_id +

idempotency_key 缓存下来。重复请求进来时直接返回历史结果，避免重复扣费、重复落库和重复触发工具。

限流与配额

传统 API 常按 QPS 限流。LLM 不行。

两个请求都是 1 次调用，但成本可能差几十倍：

- 请求 A：输入 500 Token，输出 100 Token。
- 请求 B：输入 80K Token，输出 8K Token。

如果只看请求数，B 和 A 一样。但对供应商配额、账单和延迟来说，它们完全不是一个量级。

LLM Gateway 通常要看这几层限流。

限流维度	控制对象	解决问题
用户级	单用户请求	防滥用、防脚本刷接口
租户级	团队预算	控成本、做套餐隔离
模型级	某个模型	防热门模型被打满
供应商级	OpenAI / Anthropic / DeepSeek 等	防外部依赖拖垮系统
Token 级	输入输出 Token	控真实成本和配额压力

更稳的做法是：请求发给供应商之前，先扣预算。

```
1 @dataclass
2 class TokenBudget:
3     estimated_input_tokens: int
4     reserved_output_tokens: int
5     total_reserved_tokens: int
6
7 class LLMRateLimiter(Protocol):
8     """先 estimate -> reserve，调用结束后再用真实 usage reconcile。"""
9
10    def acquire(self, tenant_id: str, user_id: str, model: str,
11    budget: TokenBudget) -> RateLimitPermit: ...
12
13    def reconcile(self, permit: RateLimitPermit, actual_usage:
14    TokenUsage) -> None: ...
15
16    def release(self, permit: RateLimitPermit) -> None: ...
```

进入 Gateway 后，先估算 input_tokens + reserved_output_tokens。用户桶、租户桶、模型桶、供应商桶都扣得动，再发请求。扣不动就排队、降级或拒绝，不要先把请求打出去再祈祷供应商别限流。

Token 估算不可能完全准，但粗估也比不估强。尤其是 RAG、长上下文、Agent 工具调用这类场景，不做预算很容易失控。

这里更推荐按四步走：estimate → reserve → 真实 usage → reconcile。先用估算值占住预算，调用结束后再用供应商返回的真实 usage 对账修正。不同供应商、不同模型的 tokenizer 和 usage 字段并不完全一致，生产里通常会先用统一近似器扣预算，再用真实 input_tokens、output_tokens 修正。如果直接按估算落库，长时间跑下来，成本和配额统计很容易积累出偏差。

成本统计

很多团队说要“降低大模型成本”，但连钱花在哪都不知道。

这不是优化，这是猜。

LLM Gateway 要记录每次调用的成本归因字段。

字段	说明
request_id	一次业务请求的唯一 ID
attempt_id	一次模型调用尝试, fallback 或重试会产生多个
tenant_id	租户或团队
user_id	用户
scene	业务场景, 比如客服、摘要、代码生成
prompt_version	Prompt 版本
provider	供应商
model_tier	路由选中的内部模型层级
model	实际调用模型
input_tokens	输入 Token
output_tokens	输出 Token
cached_tokens	命中 Prompt cache 或供应商缓存的 Token
cost	按价格快照计算的成本
price_version	成本计算使用的价格版本或生效时间
latency_ms	总延迟
ttft_ms	首 Token 延迟
fallback_used	是否发生 fallback
error_code	错误类型

有了这些字段, 排查和控成本才有抓手:

- 哪个租户成本最高?
- 哪个功能最烧 Token?
- 哪个 Prompt 版本导致输出变长?
- 哪个模型在某个场景下性价比最好?
- fallback 发生在什么时间段、什么供应商、什么模型?
- 模型升级后, 成本和质量有没有变化?

成本计算也要有版本。模型价格、缓存折扣、供应商计费项都会调整, 账单对不上时需要知道当时用的是哪份价格表。生产里不要只存一个 cost, 最好同时保存 usage 明细、价格版本和计算时间。

成本优化不会在调完一次参数后结束, 后面还要持续看数据、改路由、回放失败样本。

观测与审计

传统系统出问题, 看日志、Trace、指标。AI 系统也一样, 只是要多记录一些模型相关字段。

Cloudflare AI Gateway、LiteLLM、Kong AI Gateway 这类产品都把日志、Token、成本、错误、延迟、缓存、限流放在很显眼的位置。AI 应用出问题时, 如果只记录最终答案, 基本没法复盘。

一次模型调用的 Trace 至少应该长这样:

```
1 {
2   "request_id": "req_202605210001",
3   "attempt_id": "att_01",
4   "tenant_id": "team_java",
5   "user_id": "u_1024",
6   "scene": "knowledge_qa",
7   "prompt_version": "rag_qa-v7",
8   "provider": "openai",
9   "model_tier": "tier-balanced",
10  "model": "provider-model-id",
11  "route_reason": "scene=knowledge_qa,cost_priority=true",
12  "input_tokens": 4210,
13  "output_tokens": 612,
14  "cost": 0.0059,
15  "ttft_ms": 680,
16  "latency_ms": 4120,
17  "fallback_used": false,
18  "finish_reason": "stop"
19 }
```

但审计有一个边界: 不要无脑长期保存完整 Prompt 和完整回答。

Prompt 里可能有用户隐私、企业文档、内部代码、合同条款。生产系统需要支持脱敏、采样、留存周期、按租户配置是否保存 payload。Cloudflare AI Gateway 文档里也提供了类似控制, 例如可以配置是否采集请求和响应正文。企业内部自研时, 也应该把“是否保存原文”做成策略, 而不是默认全量落库。

一个稳妥的默认策略可以这样定:

- 元数据长期保留: usage、模型、延迟、成本、route_reason、错误码这类字段尽量留全。

- Prompt 和响应正文采样存储: 比如 1% 全量留 90 天, 其他只留 hash、长度、脱敏摘要和结构化错误。
- PII 在入口做检测和脱敏: 手机号、身份证、银行卡、邮箱、地址等字段先替换为占位符, 再进入日志链路。
- 按租户配置开关: 合同明确允许留存的租户可以保留全量; 未明确授权的租户只留元数据。
- 提供导出和删除接口: 方便满足 GDPR、数据主体请求和企业内部审计要求。

缓存与语义缓存

缓存是降本利器, 但在 LLM 场景里很容易用错。

缓存类型	做法	适合场景	风险
精确缓存	请求完全一致时返回旧结果	FAQ、固定说明、重复测试	个性化和权限场景容易错
OpenAI Prompt Caching	稳定长前缀自动命中缓存	长系统提示、稳定工具 Schema	支持模型、阈值和折扣以官方文档和价格表为准
Anthropic Prompt Caching	用 <code>cache_control</code> 标记可缓存块	长系统提示、大文档、多轮 Agent	写入和读取的计费规则要按当前价格表核对
Gemini Context Caching	通过 cached content 机制复用长上下文	长文档、视频、代码库、多轮问答	要管理缓存对象、TTL、存储成本和失效
语义缓存	语义相似的问题复用旧答案	客服 FAQ、产品说明、低风险问答	相似不等于相同, 容易答偏
结果片段缓存	缓存中间摘要、检索结果、工具结果	长文档摘要、批处理	缓存失效和版本管理复杂

客服 FAQ 这类问题很适合缓存: “怎么修改密码”“发票在哪里下载”“会员怎么退款”。这些答案稳定, 个性化少, 缓存收益明显。

但下面这些不适合随便缓存:

- 带用户权限的问题。
- 查询实时状态的问题。
- 金融、医疗、法务建议。
- 包含私密上下文的多轮对话。
- 依赖当前时间、订单状态、库存状态的问题。

语义缓存尤其要谨慎。“我的订单为什么没发货”和“我的订单能不能退款”可能语义接近, 但业务动作完全不同。缓存命中率很好看, 不代表用户体验好。

Prompt cache 也不是开了就赚。显式缓存通常要区分写入和读取; 自动缓存也会受支持模型、最小前缀长度、价格表变化影响。如果你的 system prompt、工具 Schema 或上下文每次都夹带时间戳、随机 ID、用户临时状态, 前缀一直变, 缓存命中率上不去, 成本收益就会很差。稳定内容放前面、动态内容放后面, 是使用供应商缓存时最重要的 Prompt 结构原则。

如何让你设计一个 LLM Gateway, 你会怎么做?

一个生产级 LLM Gateway 长什么样?

设计 LLM Gateway 时, 可以先拆成这些组件:

组件	职责
API Adapter	对外暴露统一 API, 兼容 OpenAI 风格请求或内部标准请求
Auth / Tenant	鉴权、租户识别、套餐和权限校验
Prompt Renderer	渲染 Prompt 模板, 记录 Prompt 版本
Token Budget Estimator	估算输入输出 Token, 判断是否超预算
Model Registry	维护模型能力、价格、上下文、供应商、状态
Router	根据场景、预算、延迟、风险选择模型
Provider Adapter	通过统一的 <code>ProviderClient</code> 接口适配各家协议差异, 包括工具调用、流式事件、usage 和错误码
Retry / Fallback	按错误类型做重试、降级和熔断
Rate Limiter	用户、租户、模型、供应商、Token 多维限流
Cost Tracker	记录 usage, 计算成本, 按租户和场景归因
Observability	输出指标、日志、Trace、告警
Audit Log	审计关键请求, 支持脱敏、留存和回放

第一版不用全部做满。建议按优先级落地：

1. 统一 API 和 Provider Adapter。
2. usage、成本、错误和延迟日志。
3. 规则路由和 Fallback。
4. Token 预算和租户配额。
5. 可观测、审计和质量回放。
6. 轻量分类器或学习型 Router。

这样每一步都有收益，也不至于一上来就把自己拖进平台工程。

请求进来后，Gateway 内部怎么跑？

一次请求在 Gateway 里通常会经历这些阶段：

1. **鉴权与租户识别**：确认用户是谁、属于哪个租户、能不能使用当前 AI 功能。
2. **判断任务场景**：从接口、业务参数或轻量分类器里得到 scene。
3. **渲染 Prompt**：根据场景选择 Prompt 模板，注入用户输入、上下文和工具 Schema。
4. **估算 Token 预算**：计算输入 Token，预留最大输出 Token。
5. **选择模型和供应商**：根据路由策略、模型状态、预算和风险等级选 primary model。
6. **执行限流和预算扣减**：按当前候选模型扣用户、租户、模型、供应商和 Token 桶。
7. **调用模型**：通过 Provider Adapter 发起同步或流式请求。
8. **解析响应**：处理文本、结构化 JSON、tool call、usage 和 finish reason。
9. **失败 Fallback**：按错误类型判断是否重试、切模型、排队或降级。
10. **记录 usage 和 trace**：写入成本、延迟、模型、供应商、Prompt 版本和错误信息。
11. **返回业务结果**：把统一响应交给业务服务。

路由策略怎么从简单演进到智能？

路由策略不要一步到位。前面提到的固定规则、级联路由、语义 / 分类路由、学习型路由、个性化路由和 Agentic 路由，对应的是一条演进路线，而不是一份“第一版全都要做”的清单。

更稳妥的节奏是：先让系统可控，再让系统省钱，最后才让系统变聪明。

阶段	对应策略	重点能力	进入下一阶段的信号
阶段一	固定模型 + 手动配置	把模型调用收口，避免 SDK 到处散落	多个场景开始共用模型，成本和延迟差异明显
阶段二	固定规则路由	按场景、租户、风险等级选模型	规则越来越多，人工维护开始吃力
阶段三	成本优先 / 级联路由	小模型先试，失败或低置信度再升级	有稳定的质量校验和可接受的额外延迟
阶段四	语义 / 分类路由	根据 Query 类型、复杂度、风险路由	有足够请求样本，可以评估分类器漂移
阶段五	质量反馈 + 成本回归	用 trace 回放模型质量和成本收益	有评测集、人工抽样或业务反馈闭环
阶段六	学习型 / 个性化 / Agentic	动态选择模型，甚至按步骤切模型	大流量、多任务、多模型，且有持续评测体系

阶段一只解决一件事：别让业务代码直连一堆供应商 SDK。客服问答走模型 A，报告生成走模型 B，代码生成走模型 C，哪怕全靠配置写死，也比散在十几个服务里强。这个阶段最重要的产物是统一入口、统一日志和统一模型名映射。

阶段二开始做固定规则路由。比如免费用户默认走小模型，企业用户复杂任务走强模型；`intent_classification` 走快模型，`legal_review` 强制走高质量模型并打上人工审核标记；主模型 429 或 P95 延迟升高时切备用供应商。这个阶段的规则应该尽量可解释，每次路由都写清楚 `route_reason`。

阶段三再考虑成本优先 / 级联路由。它不能只理解成“先便宜后昂贵”，关键在升级条件：结构化输出解析失败、分类置信度低、答案被规则校验拦下、用户请求明确进入高风险场景，才升级到更强模型。级联路由能省钱，但会增加一次推理和评估的延迟，所以更适合摘要、分类、客服 FAQ 这类能容忍几十到几百毫秒额外开销的场景；实时语音、在线协作编辑这类场景要谨慎。

阶段四引入语义 / 分类路由。语义路由可以把 query 编码成 embedding，和一组任务原型或模型 profile 做相似度匹配；分类路由可以用轻量分类器判断请求是事实型、分析型、代码型、闲聊型，或者复杂度是 low、medium、high。这里最容易踩的坑是阈值漂移：业务场景、用户表达、模型能力都会变，所以要定期抽样看误路由率。

阶段五才是真正的数据闭环。Gateway 要能回答这些问题：哪类请求小模型经常失败？哪类请求强模型和小模型质量差不多？哪个 Prompt 版本让输出变长？哪个租户的成本突然升高？这一步不一定马上训练 Router，但一定要建立评测集、线上 trace、人工抽样和质量标签。没有这些数据，后面的学习型路由就是空中楼阁。

阶段六才考虑学习型 Router、个性化 Router 和 Agentic Router。学习型 Router 用历史质量、成本、延迟训练模型选择器；个性化 Router 会考虑用户偏好和历史交互；Agentic Router 则在多轮任务里按步骤切模型，比如规划用强模型、工具参数生成用快模型、最终总结用便宜模型。这些策略的收益可能很高，但调试、隐私、成本上限和线上回放都会复杂很多。

所以，路由演进有一个很实际的判断标准：**没有 trace，不上分类器；没有评测集，不上学习型 Router；没有成本上限，不上 Agentic 路由**。先把规则路由、Fallback、usage、`route_reason` 跑稳，再谈智能化。

路由错了怎么办？

路由一定会错。

问题不在于能不能避免所有错误，而在于错了之后能不能发现、能不能兜底、能不能复盘。

常见兜底方式有这些：

问题	兜底方式
分类置信度低	走默认中强模型，或要求用户澄清
小模型输出低质量	自动升级强模型重试
高风险任务被路由到低风险链路	风险规则优先级高于成本规则
新模型上线后效果漂移	灰度、A/B、固定评测集回归
用户投诉答案错误	通过 request_id 回放 Prompt、模型、上下文和路由原因
某模型 P95 延迟升高	健康检查降低权重或临时熔断

路由日志里一定要记录 `route_reason`。不要只记录“用了哪个模型”，还要记录“为什么用它”。
例如：

```
1 {
2   "scene": "intent_classification",
3   "selected_model_tier": "tier-fast",
4   "selected_model": "provider-model-id",
5   "route_reason": "scene_rule:low_risk,cost_priority,
  ↳ estimated_tokens=320",
6   "confidence": 0.91,
7   "fallback_candidates": ["tier-nano", "tier-balanced"]
8 }
```

没有 `route_reason`，路由系统后期会很难调。

主流方案怎么选？

自研、LiteLLM、Cloudflare AI Gateway、Kong AI Gateway、Inworld Router 怎么选？

现在 LLM Gateway / Router 方案很多，别只看“支持多少模型”。选型时先看几个问题：团队技术栈是什么，合规要求有多强，流量规模多大，是否要自托管，是否已经有 API 网关，是否需要深度观测。

方案	主要优势	适合场景	不适合场景
自研轻量网关	可控、贴合业务，能和内部权限、计费、审计深度结合	有后端能力，需求明确，想从规则路由逐步演进	想快速接入大量供应商，或缺少网关维护能力
LiteLLM	多供应商接入、OpenAI 兼容格式、Proxy / SDK 生态成熟	平台团队、快速集成、多模型实验、统一入口	强合规或深度企业治理场景需要额外改造；生产使用要注意版本锁定和供应链安全
Cloudflare AI Gateway	托管入口、日志分析、缓存、限流、重试、动态路由、DLP、BYOK 等能力	已在 Cloudflare 平台上，想快速获得观测、缓存和统一入口	强自托管、私有化部署、复杂企业治理
Kong AI Gateway	企业 API 治理能力强，插件体系成熟，能结合鉴权、限流、PII 脱敏、成本治理	已有 Kong 基础设施，或需要把 AI 请求纳入企业 API 网关体系	小团队早期项目，或不想引入完整 API 网关体系
Inworld Router	条件路由、流量切分、实验和 sticky user assignment	实时语音、对话式 AI、AI 编程工具、用户分层和 A/B 测试	需要开源审计源码、私有化部署或明确企业 SLA 的场景需单独确认
LLMRouter / RouteLLM 类研究项目	路由算法丰富，适合验证复杂度路由、成本质量权衡	研究、实验、离线评估、验证路由策略	直接作为生产 Gateway，需要补齐鉴权、计费、审计、限流、观测和高可用

LiteLLM 的优势是供应商覆盖和接入速度。它更适合把不同供应商收敛到一个 OpenAI 兼容入口，再配合 Proxy 做中心化管理，例如虚拟 Key、预算、访问控制、日志和路由。
不过，LiteLLM 这类基础设施组件生产使用一定要做版本锁定、镜像固定和供应链扫描。2026 年 3 月，LiteLLM 官方通报过一次 PyPI 发布链路被污染事件，受影响版本包括 1.82.7 和 1.82.8；官方同时说明 GitHub 源码、Docker 镜像和 LiteLLM Cloud 不受影响。这类事件提醒我们：AI Gateway 往往持有大量上游 API Key，一旦依赖链被污染，影响面会比普通业务库更大。
Cloudflare AI Gateway 更像托管在边缘网络上的 AI 流量入口。如果你的系统已经在 Cloudflare 上，用它补日志、分

析、缓存、限流、重试和 Fallback 的接入成本会比较低。它的官方文档还提供动态路由、Bring Your Own Keys、请求 / 响应 DLP 扫描、日志 payload 采集开关等能力。对安全合规要求没到“必须私有化”的团队，这些能力能少做不少平台工程。

Kong AI Gateway 的定位更企业化。它适合把 LLM 流量纳入 Kong 既有的 API 治理体系里，用插件化方式处理路由、限流、审计、指标和安全策略。Kong 在成本治理上也更偏网关视角，比如基于 Token 用量或成本做限流、按成本或延迟做负载均衡、采集 LLM usage 和 cost 指标。需要注意的是，Kong 的部分 AI Gateway / AI Proxy Advanced 能力属于企业插件或需要对应授权，小团队早期接入前要先看部署和授权成本。

Inworld Router 更强调实时路由和实验能力。它通过条件路由、动态分层、流量切分和 sticky user assignment，把用户分层、任务复杂度、延迟目标、成本目标这些业务规则落到模型选择上。它适合实时语音、对话式 AI、AI 编程工具、用户分层和 A/B 测试这类场景。它不是开源路由库；如果你的要求是源码审计、完全自控发布节奏、私有化部署或明确 SLA，要按官方文档和商务条款单独确认。LLMRouter 更偏研究和算法工具箱。它能帮你理解和验证 KNN、SVM、MLP、Elo、Graph、个性化、多轮和 Agentic Router 这类策略，但如果要做生产 Gateway，还要补限流、审计、成本、权限、合规和运维能力。

选型建议

如果业务刚起步，先做轻量自研 Gateway。不要一上来买很重的平台，先把模型调用收口，至少做到日志、usage、Token 预算和 Fallback。

如果你要快速接入很多模型和供应商，优先看 LiteLLM 这类成熟统一接口。它能让团队很快从“到处写 SDK”切到“统一入口”。

如果企业已经在用 Kong，可以考虑 Kong AI Gateway。它的价值在于把 AI 流量放进已有 API 治理体系里。

如果已经重度使用 Cloudflare，可以用 Cloudflare AI Gateway 先把观测、缓存、限流和统一入口补上。

如果要做智能路由，先准备评测集和线上 trace，再谈 LLM-Router 这类学习型策略。没有数据，路由算法越复杂，越难解释。

这里的顺序不要反：**先解决工程治理，再追求智能路由。**

怎么衡量 LLM Gateway 做得好不好？

LLM Gateway 做得好不好，不能只看“接了多少模型”。模型接得多，只能说明适配层写得多，不能说明线上链路稳定。

更有用的是看下面这些数据：

指标	含义
路由命中率	请求是否进入预期模型或预期模型层级
质量通过率	输出是否通过评测、人工抽样或业务校验
Fallback 率	主链路是否稳定，备用链路是否频繁触发
平均成本	单次请求或单业务场景成本
P95 延迟	用户体验，尤其是在线交互和语音场景
TTFT	首 Token 延迟，影响流式体验
429 率	供应商限流压力
缓存命中率	缓存节省的请求和 Token
结构化解析失败率	Schema、Prompt、模型适配是否稳定
路由漂移	模型升级或流量变化后，原路由策略是否失效

这里面最容易被忽略的是“路由漂移”。

模型能力不是静态的。一个便宜模型今天不适合复杂摘要，三个月后升级了，可能已经够用。反过来，一个原本稳定的模型升级后，也可能在某类格式化任务上变差。

所以路由规则不能写完就不管。它要像 Prompt 一样有版本，像代码一样做回归测试。

总结

面试里问到大模型网关，不要只回答“统一转发模型请求”。这个说法太浅了。

更完整的回答应该是：LLM Gateway 负责把模型调用收口，统一处理模型接入、路由、Fallback、限流、Token 预

算、成本归因、日志审计和质量回放。LLM Router 只是其中负责“选哪个模型”的一部分。

第一版不用做得很重。先把模型调用从业务代码里抽出来，记录清楚每次请求用了哪个模型、花了多少 Token、有没有 Fallback、失败原因是什么。等这些数据有了，再去做更细的规则路由、成本优化和学习型 Router。

反过来，如果一开始就追求智能路由，但没有评测集、没有 trace、没有失败样本，系统只会多一个难解释的黑盒。模型调用这层越早收口，后面换模型、查成本、处理限流和复盘事故时越省事。

参考资料

- [LiteLLM Docs](#)
- [LiteLLM Security Update: Suspected Supply Chain Incident](#)
- [Cloudflare AI Gateway Docs](#)
- [Cloudflare AI Gateway Request Handling](#)
- [Cloudflare AI Gateway Fallbacks](#)
- [Cloudflare AI Gateway DLP](#)
- [Cloudflare AI Gateway BYOK](#)
- [Kong AI Gateway Docs](#)
- [Inworld Router Docs](#)
- [LLMRouter GitHub Repository](#)
- [OpenAI Prompt Caching](#)
- [Anthropic Prompt Caching](#)
- [Gemini Context Caching](#)